دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه شانزدهم: درهم‌سازی کامل و درهم‌سازی پویا

دکتر شهرام گلزاری

پاییز ۱۴۰۵

اهداف جلسه

در پایان این جلسه انتظار داریم که:

- برای مجموعه‌ای ایستا از کلیدها و توابع داده‌شده‌ی سطح اول و دوم، جدول درهم‌سازی کامل را بسازیم و حافظه‌ی کل آن را حساب کنیم.
- برای دنباله‌ای از درج‌ها در جدول پویا، اندازه‌ی جدول و هزینه‌ی هر درج را تعیین کنیم و هزینه‌ی سرشکن را به دست آوریم.

درهم‌سازی کامل

کلیدهای ایستا

«با این‌که روش درهم‌سازی به‌دلیل عمل‌کرد مطلوبش در حالت میانگین مورد استفاده قرار می‌گیرد، اگر مجموعه‌ی کلیدهای داده شده «ایستا» (static) باشند، یعنی از قبل معلوم باشند، می‌توان کاری کرد که در بدترین حالت نیز عمل‌کردی عالی داشته باشند. در برخی از کاربردها با کلیدهای ایستا سروکار داریم؛ مانند، مجموعه‌ی کلمات کلیدی در یک زبان برنامه‌نویسی، یا مجموعه‌ی اسامی موجود در یک لوح فشرده.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۶

در زنجیره‌سازی و آدرس‌دهی باز، جست‌وجو در بدترین حالت زمان $\Theta(n)$ می‌گیرد، حتی با درهم‌سازی سراسری.

تعریف درهم‌سازی کامل

«یک روش درهم‌سازی را کامل (perfect hashing) می‌نامیم اگر در بدترین حالت تعداد دسترسی‌ها به حافظه برای یک عمل جست‌وجو $O(1)$ باشد. ایده‌ی اصلی درهم‌سازی کامل ساده است: استفاده از دو سطح درهم‌سازی و یک تابع درهم‌سازی سراسری برای هر سطح.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۶

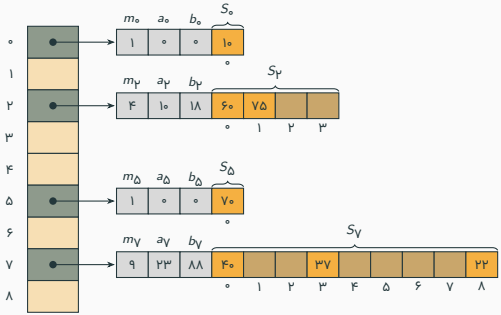
ساختار دو سطحی

«سطح اول در واقع همان درهم‌سازی زنجیره‌ای است: n کلید به‌وسیله‌ی تابع درهم‌سازی h که با دقت از خانواده‌ای از توابع درهم‌سازی سراسری انتخاب شده است به m درایه نگاشته می‌شود. به‌جای ساختن لیستی از کلیدهای فرستاده شده به خانه‌ی j، از جدول درهم‌سازی ثانویه‌ی کوچکی به نام S_j و با تابع درهم‌سازی h_j استفاده می‌کنیم. با انتخاب دقیق h_j، می‌توان تضمین کرد که هیچ برخوردی در سطح دوم رخ نمی‌دهد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۷

جست‌وجوی کلید k: محاسبه‌ی $j = h(k)$ و سپس بررسی تنها یک خانه، یعنی $S_j[h_j(k)]$.

شکل درهم‌سازی کامل



«شکل ۵-۶ مثالی از درهم‌سازی کامل که در آن کلیدهای $K = \{۱۰, ۲۲, ۳۷, ۴۰, ۶۰, ۷۰, ۷۵\}$ با دو تابع درهم‌سازی سراسری به جدول درج می‌شوند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۷ (بازترسیم شکل ۵-۶)

سطح اول مثال

«تابع درهم‌سازی اولیه $h(k) = ((ak + b) \bmod p) \bmod m$ است که در آن $a = ۳$، $b = ۴۲$، $p = ۱۰۱$ و $m = ۹$. مثلاً، $h(۷۵) = ۲$، یعنی کلید ۷۵ به درایه‌ی ۲ از جدول T نگاشته می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۷

k	۱۰	۲۲	۳۷	۴۰	۶۰	۷۰	۷۵
$۳k + ۴۲$	۷۲						
$\bmod ۱۰۱$	۷۲						
$h(k)$	۰						

کلید ۱۰ به درایه‌ی ۰ می‌رود.

سطح اول مثال

«تابع درهم‌سازی اولیه $h(k) = ((ak + b) \bmod p) \bmod m$ است که در آن $a = ۳$، $b = ۴۲$، $p = ۱۰۱$ و $m = ۹$. مثلاً، $h(۷۵) = ۲$، یعنی کلید ۷۵ به درایه‌ی ۲ از جدول T نگاشته می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۷

k	۱۰	۲۲	۳۷	۴۰	۶۰	۷۰	۷۵
$۳k + ۴۲$	۷۲	۱۰۸	۱۵۳	۱۶۲			
$\bmod ۱۰۱$	۷۲	۷	۵۲	۶۱			
$h(k)$	۰	۷	۷	۷			

کلیدهای ۲۲، ۳۷ و ۴۰ هر سه به درایه‌ی ۷ می‌روند.

سطح اول مثال

«تابع درهم‌سازی اولیه $h(k) = ((ak + b) \bmod p) \bmod m$ است که در آن $a = ۳$، $b = ۴۲$، $p = ۱۰۱$ و $m = ۹$. مثلاً، $h(۷۵) = ۲$، یعنی کلید ۷۵ به درایه‌ی ۲ از جدول T نگاشته می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۷

۷۵	۷۰	۶۰	۴۰	۳۷	۲۲	۱۰	k
۲۶۷	۲۵۲	۲۲۲	۱۶۲	۱۵۳	۱۰۸	۷۲	$۳k + ۴۲$
۶۵	۵۰	۲۰	۶۱	۵۲	۷	۷۲	mod۱۰۱
۲	۵	۲	۷	۷	۷	۰	$h(k)$

پس $n_۰ = ۱$، $n_۲ = ۲$، $n_۵ = ۱$، $n_۷ = ۳$ و بقیه‌ی درایه‌ها خالی‌اند، همان‌گونه که در شکل ۵-۶ دیده می‌شود.

سطح دوم مثال

«جدول ثانویه‌ی درهم‌سازی S_j، تمامی کلیدهای فرستاده شده به درایه‌ی j را ذخیره می‌کند. اندازه‌ی S_j برابر m_j و تابع درهم‌سازی مربوط به آن $h_j(k) = ((a_j k + b_j) \bmod p) \bmod m_j$ است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۷

j	n_j	$m_j = n_j^2$	(a_j, b_j)	مقدار h_j برای کلیدهای درایه‌ی j
۰	۱	۱	$(۰, ۰)$	$h_۰(۱۰) = ۰$
۲	۲	۴	$(۱۰, ۱۸)$	$h_۲(۶۰) = ۱۲ \bmod ۴ = ۰,\ h_۲(۷۵) = ۶۱ \bmod ۴ = ۱$
۵	۱	۱	$(۰, ۰)$	$h_۵(۷۰) = ۰$
۷	۳	۹	$(۲۳, ۸۸)$	$h_۷(۲۲) = ۸,\ h_۷(۳۷) = ۳,\ h_۷(۴۰) = ۰$

در هیچ جدول ثانویه‌ای برخورد رخ نداده است. مجموع اندازه‌های جدول‌های ثانویه $۱۵ = ۹ + ۱ + ۴ + ۱$ است. برای $n_j = ۱$: «a و b را مساوی ۰ می‌گیریم».

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۹

اندازه‌ی جدول ثانویه

«به‌منظور تضمین این ادعا، اندازه‌ی جدول درهم‌سازی S_j، m_j را برابر مجذور تعداد مقادیر فرستاده شده به جدول S_j، یا n_j^2، قرار می‌دهیم. هرچند ممکن است به نظر برسد که این رابطه‌ی درجه‌ی دو بین m_j و n_j موجب استفاده‌ی زیاد از حافظه می‌شود، اما نشان خواهیم داد که با انتخاب خوب تابع درهم‌سازی اولیه، امید ریاضی کل حافظه‌ی استفاده شده همچنان $O(n)$ خواهد ماند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۷ و ۲۸۸

انتخاب توابع

«توابع درهم‌سازی خود را از خانواده‌های سراسری توابع درهم‌سازی مربوط به بخش ۵-۵ انتخاب می‌کنیم. تابع سطح اول از خانواده‌ی $\mathcal{H}_{p,m}$ انتخاب می‌شود که در آن p عددی اول و بزرگ‌تر از تمام مقادیر داده‌هاست. مقادیری که به خانه‌ی j فرستاده می‌شوند دوباره به‌وسیله‌ی تابع درهم‌سازی h_j که از خانواده‌ی $\mathcal{H}_{p,m_j}$ انتخاب شده است، به جدول درهم‌سازی S_j با اندازه‌ی m_j ارسال می‌شوند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۸

در ادامه دو چیز را نشان می‌دهیم: می‌توان h_j را بدون برخورد پیدا کرد، و امید ریاضی کل حافظه $O(n)$ است.

قضیه‌ی ۵-۷

«اگر n مقدار را در یک جدول درهم‌سازی با اندازه‌ی $m = n^2$ با استفاده از تابع درهم‌سازی h که به‌صورت تصادفی از یک خانواده‌ی سراسری توابع درهم‌سازی انتخاب شده است ذخیره کنیم، آن‌گاه احتمال این‌که برخوردی رخ بدهد کم‌تر از $\frac{1}{2}$ است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۸

مثال

$n = 5$ کلید در جدولی با $m = 25$ درایه: $\binom{5}{2} = 10$ جفت داریم و هر جفت با احتمال حداکثر ۱/۲۵ برخورد می‌کند. پس امید تعداد برخوردها حداکثر ۰/۴ $= 10/25$ و احتمال وقوع برخورد حداکثر ۰/۴ است.

اثبات قضیه‌ی ۵-۷

اثبات. «$\binom{n}{۲}$ جفت مقدار وجود دارد که می‌توانند با هم برخورد داشته باشند. اگر h به‌صورت تصادفی از بین خانواده‌ی سراسری $\mathcal{H}$ از توابع درهم‌سازی انتخاب شده باشد، هر جفت با احتمال $\frac{۱}{m}$ برخورد خواهد داشت. فرض کنید X متغیری تصادفی است که تعداد برخوردها را می‌شمارد. با فرض $m = n^۲$، امید ریاضی تعداد برخوردها برابر است با

$$E[x] = \binom{n}{۲} \cdot \frac{۱}{n^۲} = \frac{n^۲ - n}{۲} \cdot \frac{۱}{n^۲} < \frac{۱}{۲}.$$

با استفاده از نامساوی مارکوف خواهیم داشت: $Pr\{X \geq t\} \leq E[x]/t$ که اگر t را برابر ۱ قرار دهیم، اثبات کامل می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۸

یافتن تابع بدون برخورد

«از توضیح داده شده در قضیه‌ی ۵-۷ که $m = n^۲$ است، نتیجه می‌گیریم برای تابعی که به‌صورت تصادفی از $\mathcal{H}$ انتخاب شده باشد، احتمال برخورد نداشتن، بیش‌تر از احتمال برخورد داشتن است. بنابراین با داشتن مجموعه‌ی K با n عنصر، چون K ثابت است، به‌راحتی می‌توانیم با چند انتخاب تصادفی، تابع درهم‌سازی h را پیدا کنیم که هیچ برخوردی نداشته باشد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۸ و ۲۸۹

مثال

اگر h را مستقل از هم و به‌تصادف انتخاب کنیم، هر انتخاب با احتمال کم‌تر از $\frac{۱}{۲}$ برخورد دارد. پس احتمال این‌که ۳ انتخاب پیاپی همگی برخورد داشته باشند کم‌تر از $\frac{۱}{۸}$ و برای t انتخاب کم‌تر از ۲^{-t} است.

روش دو سطحی

«اما هنگامی که n بزرگ باشد، یک جدول درهم‌سازی با اندازه‌ی $m = n^۲$ زیاد از حد بزرگ است. بنابراین از روش دوسطحی درهم‌سازی استفاده می‌کنیم، و از قضیه‌ی ۵-۷ تنها در درهم‌سازی مقادیر درون هر خانه بهره می‌بریم. (...) سپس اگر n_j کلید به درایه‌ی j فرستاده شود، برای داشتن ارجاع‌هایی بدون برخورد و در زمان ثابت، از جدول درهم‌سازی ثانویه‌ی S_j با اندازه‌ی $m_j = n_j^۲$ استفاده می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۹

برای $n = ۱۰^۶$ کلید، یک جدول تک‌سطحی با $m = n^۲$ به $۱۰^{۱۲}$ درایه نیاز دارد.

خطر حافظه‌ی زیاد

«اکنون به اثبات این‌که کل حافظه‌ی استفاده شده $O(n)$ است برمی‌گردیم. چون اندازه‌ی m_j مربوط به جدول ثانویه‌ی S_j رشدی درجه دو نسبت به n_j دارد، این خطر وجود دارد که مقدار کل حافظه‌ی استفاده‌شده زیاد باشد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۹

مثال

اگر تابع سطح اول همه‌ی n کلید را به یک درایه بفرستد، آن درایه به جدول ثانویه‌ای با n^2 خانه نیاز دارد. اگر کلیدها به‌طور مساوی پخش شوند و $m = n$، هر $n_j = 1$ و مجموع اندازه‌ها n است.

قضیه‌ی ۵-۸

حافظه‌ی جدول سطح اول و پارامترهای m_j، a_j و b_j برای $m = n$ از $O(n)$ است. پس کافی است مجموع $m_j = n_j^۲$ را کران‌دار کنیم.

قضیه‌ی ۵-۸

«اگر تعداد n کلید را به‌وسیله‌ی تابع درهم‌سازی h که به‌تصادف از یک خانواده‌ی سراسری از توابع درهم‌سازی انتخاب شده است، در یک جدول درهم‌سازی با اندازه‌ی $m = n$ ذخیره کنیم، آن‌گاه

$$E\left[\sum_{j=۰}^{m-۱} n_j^۲\right] < ۲n$$

که در آن n_j تعداد کلیدهایی است که به درایه‌ی j نگاشته شده‌اند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۹

اثبات قضیه‌ی ۵-۸

اثبات. «کار را با اتحاد زیر، که برای تمام اعداد صحیح غیر منفی درست است، شروع می‌کنیم:
«$a^{۲} = a + ۲\binom{a}{۲}$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۹

$$E\Big[\sum_{j=۰}^{m-۱} n_j^{۲}\Big] = E\Big[\sum_{j=۰}^{m-۱} \Big(n_j + ۲\binom{n_j}{۲}\Big)\Big] = E\Big[\sum_{j=۰}^{m-۱} n_j\Big] + E\Big[\sum_{j=۰}^{m-۱} \binom{n_j}{۲}\Big]$$
$$= E[n] + E\Big[\sum_{j=۰}^{m-۱} \binom{n_j}{۲}\Big] = n + E\Big[\sum_{j=۰}^{m-۱} \binom{n_j}{۲}\Big].$$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۰

برای نمونه $۳^{۲} = ۳ + ۲ \cdot ۳$: سه کلید در یک درایه، ۳ جفت برخورد می‌سازند.

پایان اثبات قضیه‌ی ۵-۸

«مقدار $\sum_{j=۰}^{m-۱} \binom{n_j}{۲}$ برابر تعداد کل برخوردهاست. از خواص درهم‌سازی سراسری و از آن‌جا که $m = n$ است، نتیجه می‌گیریم که امید ریاضی این مقدار دست بالا برابر است با

$$\binom{n}{۲}\frac{۱}{m} = \frac{n(n-۱)}{۲m} = \frac{n-۱}{۲}.$$

پس

$$E\Big[\sum_{j=۰}^{m-۱} n_j^۲\Big] \le n + ۲\,\frac{n-۱}{۲} = ۲n - ۱ < ۲n.$$

»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۰

در مثال شکل ۵-۶، $\sum \binom{n_j}{۲} = ۰ + ۱ + ۰ + ۳ = ۴$ و $\sum n_j^۲ = ۷ + ۲ \cdot ۴ = ۱۵$.

نتیجه‌ی ۵-۳

«اگر بر اساس درهم‌سازی کامل، تعداد n کلید را به‌وسیله‌ی تابع درهم‌سازی h که به‌تصادف از یک خانواده‌ی سراسری توابع درهم‌سازی انتخاب شده است، در یک جدول درهم‌سازی با اندازه‌ی $m = n$ ذخیره کنیم و (برای $j = ۰, ۱, \ldots, m - ۱$) اندازه‌ی هریک از جدول‌های ثانویه را $m_j = n_j^۲$ قرار دهیم، آن‌گاه امید ریاضی حافظه‌ی مورد نیاز برای کل جدول‌های ثانویه کم‌تر از $۲n$ است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۰

اثبات. «از آن‌جا که برای $j = ۰, ۱, \ldots, m - ۱$، $m_j = n_j^۲$ است و با استفاده از قضیه‌ی ۵-۸ خواهیم داشت:

$$E\Big[\sum_{j=۰}^{m-۱} m_j\Big] = E\Big[\sum_{j=۰}^{m-۱} n_j^۲\Big] < ۲n, \qquad (۶\text{-}۵)$$

که اثبات را کامل می‌کند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۹۰ و ۲۹۱

نتیجه‌ی ۵-۴

«اگر بر اساس درهم‌سازی کامل، n کلید را به‌وسیله‌ی تابع درهم‌سازی h که به‌تصادف از یک خانواده‌ی سراسری توابع درهم‌سازی انتخاب شده است، در یک جدول درهم‌سازی با اندازه‌ی $m = n$ ذخیره کنیم و (برای $j = ۰, ۱, \ldots, m - ۱$) اندازه‌ی هریک از جدول‌های ثانویه را برابر $m_j = n_j^۲$ قرار دهیم، آن‌گاه احتمال این‌که میزان کل حافظه‌ی استفاده‌شده برای توابع درهم‌سازی ثانویه از $۴n$ بیش‌تر شود، کم‌تر از $\frac{۱}{۲}$ است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۱

اثبات. با نامساوی مارکوف و $X = \sum_{j=۰}^{m-۱} m_j$ و $t = ۴n$:

$$Pr\Big\{ \sum_{j=۰}^{m-۱} m_j \geq ۴n \Big\} \leq \frac{E\big[\sum_{j=۰}^{m-۱} m_j \big]}{۴n} < \frac{۲n}{۴n} = \frac{۱}{۲}.$$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۱

مثال حافظه‌ی کل

«با استفاده از نتیجه‌ی ۵-۳ می‌بینیم که با امتحان کردن چند تابع درهم‌سازی که به‌تصادف از خانواده‌ی سراسری توابع انتخاب شده باشند، به سرعت می‌توانیم تابعی پیدا کنیم که مقدار مناسبی حافظه استفاده کند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۱

مثال

$n = ۱۰۰۰$ کلید و $m = n$. امید مجموع اندازه‌های جدول‌های ثانویه کم‌تر از ۲۰۰۰ است و هر تابع h با احتمال کم‌تر از $\frac{۱}{۲}$ مجموعی دست‌کم ۴۰۰۰ می‌دهد. با ۱۰ انتخاب مستقل، احتمال این‌که هیچ‌کدام مجموعی کم‌تر از ۴۰۰۰ ندهد کم‌تر از $۰/۰۰۱ < ۲^{-۱۰}$ است.

درهم‌سازی پویا

اندازه‌ی نامعلوم

در روش‌هایی که تا کنون دیدیم، اندازه‌ی جدول از پیش و بر اساس بیشینه‌ی تعداد عناصر تعیین می‌شد.

«در بسیاری از کاربردها، تعداد عناصر از قبل مشخص نیست و هر لحظه با اعمال درج و حذف این تعداد تغییر می‌کند. بنابراین نمی‌توان کران بالای خوبی برای اندازه‌ی جدول درهم‌سازی پیش‌بینی کرد که حافظه تلف نشود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۲

مثال

اگر جدولی برای $۱۰^۶$ عنصر ساخته شود و در عمل فقط ۱۰۰ عنصر درج شود، بیش از ۹۹/۹ درصد درایه‌ها بی‌استفاده می‌مانند. اگر جدول کوچک ساخته شود، با رسیدن عناصر بیش‌تر جدول پر می‌شود.

جدول پویا

«در روش «درهم‌سازی پویا» (dynamic hashing) اندازه‌ی جدول درهم‌سازی برحسب نیاز زیاد و کم می‌شود. در این بخش نشان می‌دهیم که اگر زمان زیاد و کم شدن اندازه‌ی جدول را به‌درستی تعیین کنیم، و در صورت انتخاب تابع درهم‌سازی یک‌نوا، هزینه‌ی سرشکن‌شده‌ی هر یک از اعمال حذف و درج هنوز $O(1)$ باقی می‌ماند. در تحلیل این داده‌ساختار از روش تابع پتانسیل که در بخش ۳-۷-۲ گفته شد استفاده می‌کنیم.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۲

اعمال جدول پویا

اعمالی که روی جدول انجام می‌شوند:

- «درج ساده: جدول جای خالی دارد و عنصر مانند قبل در جدول درج می‌شود.»
- «درج با گسترش (expansion): قبل از درج، جدول پر است. در این‌صورت اندازه‌ی جدول دو برابر می‌شود، عناصر موجود به جدول جدید منتقل می‌شوند و سپس عنصر مورد نظر در جدول جدید درج می‌شود.»
- «حذف ساده: حذفی که موجب تغییر در اندازه‌ی جدول نمی‌شود.»
- «حذف و فشرده‌سازی (compaction): پس از حذف نسبت تعداد عناصر به‌اندازه‌ی جدول کم می‌شود و لازم است اندازه‌ی جدول نصف شود و همه‌ی عناصر موجود به جدول جدید منتقل شوند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۲

نمادهای تحلیل

«برای تحلیل، نمادهای زیر را تعریف می‌کنیم:

- c_i هزینه‌ی عمل iام،
- s_i اندازه‌ی جدول پس از عمل iام،
- n_i تعداد عناصر جدول پس از عمل iام، و
- $0 \leq \alpha_i = \frac{n_i}{s_i} \leq 1$ ضریب بار پس از عمل iام.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۹۲ و ۲۹۳

برای نمونه اگر پس از عمل ۱۰ام جدولی با ۱۶ درایه و ۱۰ عنصر داشته باشیم، $s_{10} = 16$، $n_{10} = 10$ و $\alpha_{10} = 0/625$.

تحلیل درج

فقط درج

«اگر در درهم‌سازی پویا فقط درج داشته باشیم، فرض می‌کنیم که اندازه‌ی جدول همیشه توانی از ۲، و در ابتدا برابر صفر است. رویه‌ی TABLE-INSERT کار درج در این جدول را انجام می‌دهد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۳

«در پیاده‌سازی، فرض می‌کنیم که T جدول درهم‌سازی، $T[table]$ مؤلفه‌ی جدول، $T[size]$ اندازه‌ی جدول، و $T[num]$ تعداد عناصر موجود در جدول T باشد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۲

در شبه‌کد با نمادگذاری CLRS به‌جای آن‌ها $T.table$، $T.size$ و $T.num$ می‌نویسیم.

```
TABLE-INSERT(T, x)
 1  if T.size == 0
 2      allocate T.table with one slot
 3      T.size = 1
 4  if T.num == T.size
 5      allocate new-table with 2 · T.size slots
 6      insert all items from T.table
            into new-table
 7      free T.table
 8      T.table = new-table
 9      T.size = 2 · T.size
10  insert x into T.table
11  T.num = T.num + 1
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۳، با نمادگذاری *CLRS*

شرح رویه

«اگر اندازه‌ی جدول صفر باشد، جدولی با یک درایه ایجاد می‌شود و x را در آن قرار می‌دهد. وگرنه، اگر تعداد عناصر موجود در جدول برابر اندازه‌ی جدول بود، یعنی قبل از درج، $\alpha_{i-1} = 1$، در آن صورت «درج با گسترش» انجام می‌شود. (...) سپس همه‌ی عناصر موجود در table با تابع درهم‌سازی جدید (با احتساب اندازه‌ی جدید جدول) در new-table درج می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۳

- خط‌های ۱ تا ۳: ساخت جدول یک‌درایه‌ای برای نخستین درج.
- خط‌های ۴ تا ۹: گسترش، که هزینه‌ی آن متناسب با تعداد عناصر منتقل‌شده است.
- خط‌های ۱۰ و ۱۱: درج واقعی با هزینه‌ی ثابت.

درج $x_1, x_2, \ldots, x_9$ در جدول پویای خالی با رویه‌ی TABLE-INSERT:



درج x_1: اندازه‌ی جدول صفر است و جدولی با یک درایه ساخته می‌شود. $s_1 = 1$ و $c_1 = 1$.

درج $x_1, x_2, \ldots, x_9$ در جدول پویای خالی با رویه‌ی TABLE-INSERT:



درج x_2: جدول پر است ($\alpha_1 = 1$). گسترش به اندازه‌ی ۲، انتقال ۱ عنصر و درج. $c_2 = 2$.

درج $x_1, x_2, \ldots, x_9$ در جدول پویای خالی با رویه‌ی TABLE-INSERT:



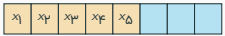
درج x_3: گسترش به اندازه‌ی ۴، انتقال ۲ عنصر و درج. $c_3 = 3$.

درج $x_۱, x_۲, \ldots, x_۹$ در جدول پویای خالی با رویه‌ی TABLE-INSERT:



درج $x_۴$: درج ساده. $c_۴ = ۱$.

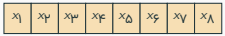
درج $x_1, x_2, \ldots, x_9$ در جدول پویای خالی با رویه‌ی TABLE-INSERT:



درج x_5: گسترش به اندازه‌ی ۸، انتقال ۴ عنصر و درج. $c_5 = 5$.

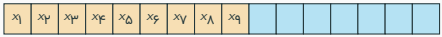
ردیابی درج‌ها

درج $x_۱, x_۲, \ldots, x_۹$ در جدول پویای خالی با رویه‌ی TABLE-INSERT:



درج‌های $x_۶$ تا $x_۸$: درج ساده. $c_۶ = c_۷ = c_۸ = ۱$.

درج $x_1, x_2, \ldots, x_9$ در جدول پویای خالی با رویه‌ی TABLE-INSERT:



درج x_9: گسترش به اندازه‌ی ۱۶، انتقال ۸ عنصر و درج. $c_9 = 9$.

هزینه‌ی کل ۹ درج: $1+2+3+1+5+1+1+1+9 = 24$. فقط درج‌های ۳ و ۹ هزینه‌ای بیش از ۲ دارند.

هزینه‌ی بدترین حالت

«به‌نظر می‌رسد که هزینه‌ی انجام درج، به‌علت هزینه‌ی گسترش یک جدول به‌اندازه‌ی n، زیاد است، در بدترین حالت $O(n)$ است. این تحلیل در بدترین حالت درست است، اما روشن است که بعد از هر گسترش جدول، تعداد زیادی از درج‌ها با همان هزینه‌ی $O(1)$ انجام می‌شود (البته با فرض آن‌که تابع درهم‌سازی ساده و یک‌نواست). این مشاهده ما را به تحلیل سرشکنی که در بخش ۳-۷ گفته شد، هدایت می‌کند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۴

ضرب کران بدترین حالت در تعداد درج‌ها، کران $O(n^2)$ را برای n درج می‌دهد که کران محکمی نیست.

«جدول زیر مثالی از نحوه‌ی گسترش جدول برای درج‌های متوالی است که در آن اندازه‌ی جدول و تعداد گسترش‌های انجام شده از ابتدا، برای درج iام نشان داده شده است.»

i	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	...
s_i	۱	۲	۴	۴	۸	۸	۸	۸	۱۶	۱۶	۱۶	۱۶
تعداد گسترش‌ها	۱	۲	۳	۳	۴	۴	۴	۴	۵	۵	۵	۵

«روشن است که تعداد کل گسترش‌ها برای n درج، برابر $\lceil \lg n \rceil + ۱$ است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۴

هزینه‌ی کل درج‌ها

«هزینه‌ی درج iام اگر گسترشی در کار نباشد برابر ۱ است و اگر گسترش انجام شود برابر $n_{i-1}+1$ است، چرا که قبل از درج آخر باید n_{i-1} عنصر موجود در جدول قبلی به جدول جدید منتقل شوند. یعنی

$$c_i = \begin{cases} 1 & \text{درج ساده} \\ n_{i-1}+1 & \text{درج با گسترش} \end{cases}$$

بنابراین مجموع هزینه‌ی n بار عمل درج برابر است با

$$\sum_{i=1}^{n} c_i = n + \sum_{k=0}^{\lceil \lg n \rceil} 2^k = n + \frac{2^{\lceil \lg n \rceil + 1} - 1}{2 - 1} \leq 3n.$$

یعنی هزینه‌ی سرشکن‌شده‌ی هر درج حداکثر ۳ واحد یا $O(1)$ است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۴

مثال روش انبوهه

برای $n = ۹$ درج (اسلاید ردیابی):

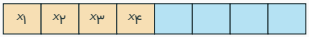
- گسترش‌ها در درج‌های ۱، ۲، ۳، ۵ و ۹ رخ می‌دهند، یعنی $\lceil \lg ۹ \rceil + ۱ = ۵$ گسترش.
- هزینه‌ی انتقال در این گسترش‌ها $۰, ۱, ۲, ۴, ۸$ است و هزینه‌ی کل $۲۷ = ۳ \cdot ۹ \leq ۲۴ = ۱۵ + ۹$.
- میانگین هزینه‌ی هر درج $۲۴/۹ \approx ۲٫۶۷$ است.

«از تحلیل فوق حدس می‌زنیم که به‌روش حساب‌داری باید به‌ازای هر درج ۳ ریال خرج کنیم: ۱ ریال آن‌را صرف خود عمل درج می‌کنیم و ۱ ریال دیگر را بر روی عنصر درج‌شده قرار می‌دهیم. سپس با استقرا نشان می‌دهیم که می‌توانیم ۱ ریال باقی‌مانده را طوری بر روی یک عنصر «نظیر» عنصر درج‌شده قرار دهیم تا در ابتدای هر گسترش، هر کدام از عناصر یک ریال داشته باشند (یعنی گسترش بتواند مجانی انجام شود) (...)»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۹۴ و ۲۹۵

مثال روش حسابداری

جدولی با ۸ درایه، درست پس از گسترش، با ۴ عنصر و بدون ریال:



$k = ۳$: اندازه‌ی جدول $۸ = ۲^{۳}$ و تعداد عناصر $۴ = ۲^{۲}$.

مثال روش حسابداری

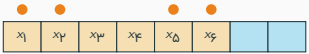
جدولی با ۸ درایه، درست پس از گسترش، با ۴ عنصر و بدون ریال:



درج x_5 با ۳ ریال: ۱ ریال برای درج، ۱ ریال روی x_5 و ۱ ریال روی عنصر نظیر آن، x_1.

مثال روش حسابداری

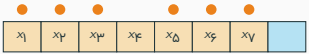
جدولی با ۸ درایه، درست پس از گسترش، با ۴ عنصر و بدون ریال:



درج $x_۶$: یک ریال روی $x_۶$ و یک ریال روی $x_۲$.

مثال روش حساب‌داری

جدولی با ۸ درایه، درست پس از گسترش، با ۴ عنصر و بدون ریال:



درج x_7: یک ریال روی x_7 و یک ریال روی x_3.

مثال روش حسابداری

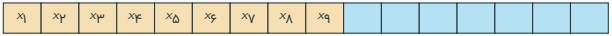
جدولی با ۸ درایه، درست پس از گسترش، با ۴ عنصر و بدون ریال:



درج x_8: جدول پر است و هر ۸ عنصر یک ریال دارند.

مثال روش حسابداری

جدولی با ۸ درایه، درست پس از گسترش، با ۴ عنصر و بدون ریال:



درج x_9: گسترش به ۱۶ درایه. هزینه‌ی انتقال ۸ عنصر با ۸ ریال ذخیره‌شده پرداخت می‌شود و x_9 مانند x_5 با ۳ ریال درج می‌شود.

اثبات روش حسابداری

«فرض کنید که بلافاصله پس از یک گسترش (و قبل از درج واقعی عنصر) اندازه‌ی جدول برابر ۲^k و تعداد عناصر موجود $۲^{k-۱}$ باشد. بنابراین برای درج همین تعداد عنصر تا گسترش بعدی، فضا موجود است. چنان‌چه گفته شد، هنگام درج هر کدام از این عناصر ۱ ریال بر روی آن و ۱ ریال بر روی یکی از عناصر موجود در جدول قرار می‌دهیم. بنابراین وقتی این جدول پر می‌شود، هر عنصر جدول ۱ ریال دارد (...)»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۵

پس هزینه‌ی گسترش از ریال‌های ذخیره‌شده پرداخت می‌شود و هزینه‌ی سرشکن هر درج ۳ است.

«برای این مسئله تابع پتانسیل Φ تعریف می‌کنیم که $\Phi(D_i) \geq ۰$. در این‌صورت، اگر عمل i موجب گسترش شود، $\Phi(D_{i-۱}) \geq s_{i-۱}$. تعریف می‌کنیم $\Phi(D_i) = ۲n_i - s_i$. به‌وضوح $\Phi(D_i) \geq ۰$ و اگر عمل i موجب گسترش شود، داریم

$$s_{i-۱} = n_{i-۱}, \quad \Phi(D_{i-۱}) = s_{i-۱}, \quad s_i = ۲s_{i-۱}, \quad n_i = n_{i-۱} + ۱.$$

بنابراین مطابق تعریف ۳-۴۰ از بخش ۳-۷-۲،»

$$\begin{aligned}
\hat{c}_i &= ۱ + \Phi(D_i) - \Phi(D_{i-۱}) \\
&= n_{i-۱} + ۱ + [۲n_i - s_i] - [۲n_{i-۱} - s_{i-۱}] \\
&= n_{i-۱} + ۱ + [۲n_{i-۱} + ۲ - ۲s_{i-۱}] - [۲n_{i-۱} - s_{i-۱}] = n_{i-۱} - s_{i-۱} + ۱ + ۲ = ۳.
\end{aligned}$$

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۵

درج عادی

«برای عمل درج عادی iام، $s_i = s_{i-1}$ و $n_i = n_{i-1} + 1$، پس

$$\begin{aligned}\hat{c}_i &= 1 + \Phi(D_i) - \Phi(D_{i-1}) = 1 + [2n_i - s_i] - [2n_{i-1} - s_{i-1}] \\ &= 1 + [2n_{i-1} + 2 - s_{i-1}] - [2n_{i-1} - s_{i-1}] = 1 + 2 = 3,\end{aligned}$$

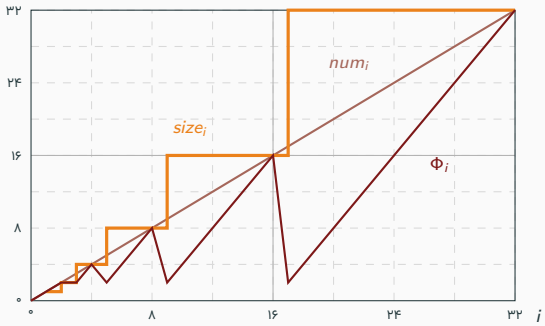
یعنی در هر صورت هزینه‌ی سرشکن‌شده‌ی هر درج حداکثر ۳ واحد یا $O(1)$ است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۹۵ و ۲۹۶

مثال

در ردیابی، پیش از درج x_9: $\Phi(D_8) = 2 \cdot 8 - 8 = 8$ و پس از آن $\Phi(D_9) = 2 \cdot 9 - 16 = 2$. پس $\hat{c}_9 = 9 + 2 - 8 = 3$. برای درج ساده‌ی x_6: $\hat{c}_6 = 1 + (12 - 8) - (10 - 8) = 3$.

نمودار پتانسیل



«شکل ۵-۷ منحنی‌های اندازه، تعداد عناصر و تابع پتانسیل به‌ازای درج‌های مختلف در جدول درهم‌سازی پویا، در حالتی‌که عمل حذف نداشته باشیم.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۹۶ (بازترسیم شکل ۵-۷)

منابع

منابع

- محمد قدسی، داده‌ساختارها و مبانی الگوریتم‌ها، انتشارات فاطمی، چاپ ۱۳۹۷